CRASH RECOVERY

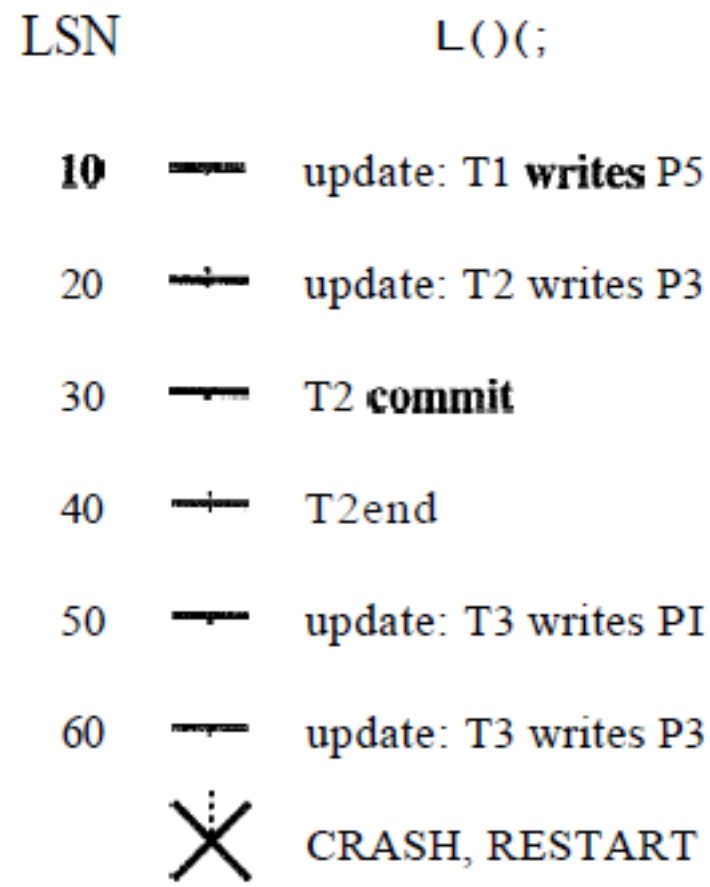
Concepts

- ARIES recovery algorithm
- LOG
- Recovery related data structures
- Write-Ahead Logging Protocol
- Checkpointing

ARIES recovery algorithm

- Restart proceeds in 3 phases
  - Analysis (identifies dirty pages and active xns)
  - Redo (repeats all actions acc to logs)
  - Undo (undoes all actions of xns that did not commit)

EXAMPLE SCHEDULE



ure 18.1 Execution History with a Crash

After restart:

Analysis Phase identifies T1 and T3 as active transactions, thus to be undone. P2 has committed, so all its actions therefore to be written to disk, P1, P3 and P5 identified as Dirty pages.

Redo Phase: All the updates (including those of TI and *T3)* are reapplied in the order shown during the Redo phase.

Undo Phase: Finally, the actions of TI and *T3* are undone in reverse order during the Undo phase; that is, *T3's* write of *P3* is undone, T3's write of P1 is undone, and then TI ‘s write of *P5* is undone.

ARIES recovery algorithm

- Three principles
  - Write-ahead logging (logs in stable storage)
  - Repeating history during Redo
  - Logging changes during Undo (to avoid repeated action on repeated restarts)

The Log

- History of actions executed by DBMS
- File of records stored in stable storage
- Multiple copies at different disks
- Log tail
- lSN (log sequence number-unique id)
- PageLSN

Actions

Log record is written for following actions

- Updating a page
- Commit
- Abort
- End
- Undoing an update

- Every log record has certain fields: <prevLSN, transID, type>

For update log record, additional fields:

- <prevLSN, transID, type, pageid, length, offset, before-image, after-image>

Figure 18.2 Contents of an Update Log Record

Compensation log record (CLR)

- CLR is written just before the change recorded in an update log record *U* is undone.

For CLR log record, additional fields:

- <prevLSN, transID, type, pageid, length, offset, before-image, after-image, undoNextLSN>
- undoNextLSN is the LSN of the next log record that is to be undone for the transaction that wrote update record U.
- Unlike an update log record, CLR describes an action tat will never be undone ie we never undo an undo action.
- A CLR may be written to stable storage (following WAL, of course) but the undo action it describes may not yet been written to disk when the system crashes again. In this case, the undo action described in the CLR is reapplied during the Redo phase, just like the action described in update log records.
- For these reasons, CLR contains the information needed to reapply, or redo the change described but not to reverse it.

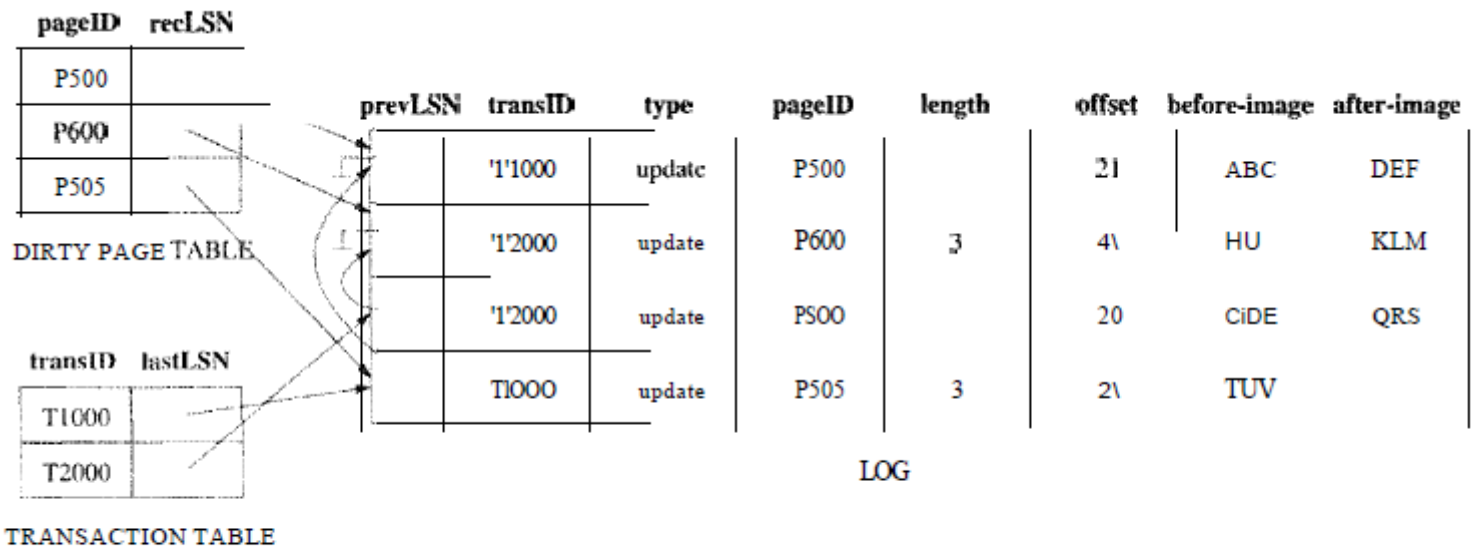


Figure 18.3 Instance of Log and Ttansaction Table

Consider the follc)\ving silupic exarnple. Transaction TIOOO changes the value of bytes 21 to 23 011 page P500 frorn 'ABC' to 'DEF', transaction 'T2000 changes 'HIJ' to 'KLM' on page P600, transaction $T2000$ changes bytes 20 through 22 fronl 'GDE' to 'QRS' on page $P500$, then transaction T1000 changes 'TUV' to 'WXY' on pageP505. The dirty page table, the transaction table,[3] and

Other recovery related data structures

- Transaction table (tid, lastLSN, status)
- Dirty page table (pageid, recLSN)

- During normal operation, these are maintained by the transaction manager and the buffer manager, respectively, and during restart after a crash, these tables are reconstructed in the Analysis phase of restart.

Checkpointing

A checkpoint is like a snapshot of the DBMS state, and by taking checkpoints periodically, as we will see, the DBl\1S can reduce the alnount of work to be done during restart in the event of a subsequent crash.

Checkpointing in ARIES has three steps. First, a begin_checkpoint record is written to indicate when the checkpoint starts. Second, an end_checkpoint record is constructed, including in it the current contents of the transaction table and the dirty page table, and appended to the log. The third step is carried out after the end_checkpoint record is written to stable storage: A special master record containing the LSN of the *begirLcheckpoint* log record is written to a known place on stable storage. While the end_checkpoint record is being constructed, the DBMS continues executing transactions and writing other log records; the only guarantee we have is that the transaction table and dirty page table are accurate *as of the time of the begin_checkpoint record*

ARIES phases

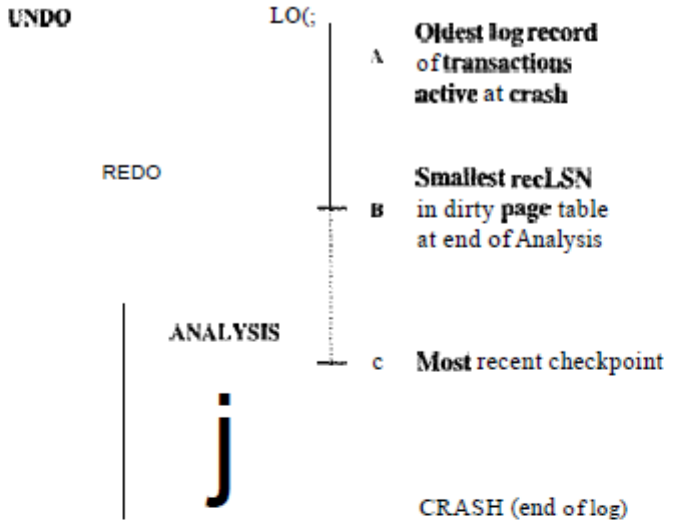


Figure 18.4 Three Phases of Restart in ARIES

The Analysis phase begins by examInIng the rnost recent begin_checkpoint record, whose LSN is denoted C in Figure 18.4, and proceeds forward in the log until the last log record. '1'he Redo phase follows Analysis and redoes all changes to any page that Illight have been dirty at the tirl1e of the crash; this set of pages and the starting point for Redo (the srnallest recLSN of any dirty page) are deterrnined during Analysis. 'The Undo phase follows Redo and undoes the changes of all transactions active at the tirne of the crash; again, this set of transactions is identified during the Analysis phase. Note that Redo reapplies changes in the order in which they were originally carried out; Undo reverses changes in the opposite order, reversing the lllost recent change first.

REDO Phase

rrhe R,edo phase begins with the log record that has the srnallest recLSN of all pages in the dirty page table constructed by the Analysis pass because this log record identifies the oldest update that rnay not have been written to disk prior to the crash. Starting frorn this log record, R,edo scans forward until the end of the log. For each redoable log record (update or CLR) encountered, Redo checks whether the logged action HUlst be redone. The action rnust be redone unless one of the follo\ving conditions holds:

- The affected page is not in the dirty page table.
- rrhe affected page is in the dirty page table, but the recLSN for the entry is *greater than* the LSN of the log record being checked.
- 1'he pageLSN (stored on the page, which rnust be retrieved to check this condition) is *greater than or equal* to the LSN of the log record being checked.

If the logged action lllust be redone:

1. The logged action is reapplied.
2. The pageLSN on the page is set to the LSN of the redone log record. No additional log record is written at this time.

UNDO Phase

Undo begins with the transaction table constructed by the .Analysis phase, which identifies all transactions active at the time of the crash, and includes the LSN of the 1110st recent log record (the lastLSN field) for each such transaction. Such transactions are called loser transactions. All actions of losers IllUst be undone, and further, these actions rnust be undone in the reverse of the order in which they appear in the log.

Consider the set of lastLSN values for all loser transactions. Let us call this set ToUndo. Undo repeatedly chooses the largest (Le., rnost recent) LSN value in this set and processes it, until rro1Jndo is ernpty. To process a log record:

1. If it is a CLR and the undoNextLSN value is not *null*, the undoNextLSN value is added to the set ToUndo; if the undoNextLSN is *null*, an end record is written for the transaction because it is cornpletely undone, and the CLR, is discarded.
2. If it is an update record, a CLR, is written and the corresponding a,ction is undone, as described in Section 18.2, and the prevLSN value in the update log record is added to the set ToUndo.

When the set rroUndo is empty, the 1Jndo phase is cornplete. I{estart is no\v cornplete, and the systenl can proceed with nonnal operations.

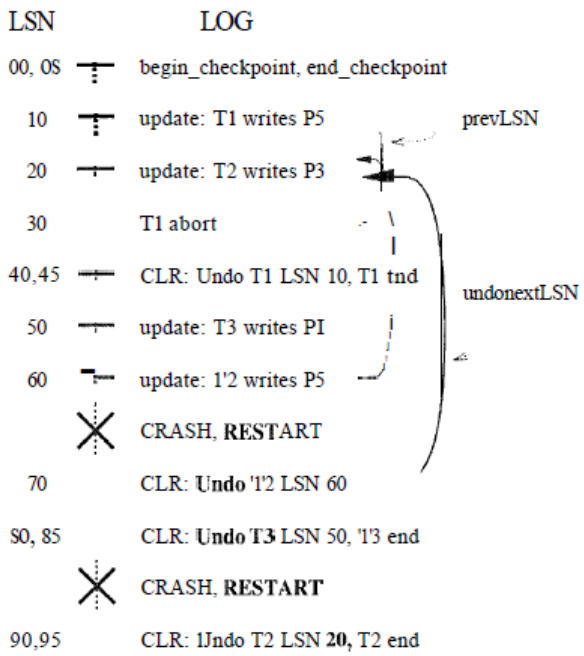


Figure 18.5 Example of Undo with Repeated Crashes